Table of contents

Complex Disease Models

Simon Frost 2026-05-14

Introduction

The expanded edge-based formulation is not limited to SIR — it handles arbitrary disease progression graphs. Each disease stage that can transmit infection gets its own edge-level probability φ, and the package automatically constructs the full system of ODEs.

This vignette demonstrates:

- SEIR: adding a latent (exposed) period
- SIS: endemic dynamics without lasting immunity
- Multi-stage infections: acute and chronic infectious periods with different transmission rates

Setup

```
using EdgeBasedModels
using ModelingToolkit
using OrdinaryDiffEq
using Symbolics
using Plots
```

```
@parameters β γ κ
pgf = poisson_pgf(κ)
κ_val = 5.0
ψ(x) = exp(κ_val * (x - 1))
tspan = (0.0, 40.0)
γ_val = 0.25
R0_target = 2.0
seed_fraction = 0.01
T_val = R0_target / κ_val
β_val = T_val * γ_val / (1 - T_val)
```

```
0.16666666666666669
```

SEIR model

> [!NOTE]
>
> $R_0 = 2$ anchor. Generic SIR/SIS comparisons use $\gamma = 0.25$, 1% seeding, and Poisson $\kappa = 5$, giving $T = 2/5$ and per-edge $\beta = 1/6$. The SEIR latent period changes timing but not this transmissibility-based R_0.

The SEIR model adds an exposed (latent) class E between S and I. Exposed individuals are infected but not yet infectious. The transition rate from E to I is σ (so the mean latent period is $1/\sigma$).

$$S \xrightarrow{\text{infection}} E \xrightarrow{\sigma} I \xrightarrow{\gamma} R$$

```
@parameters σ
model_seir = build_seir(pgf, σ, β, γ)
eqs = ModelingToolkit.equations(model_seir.system)
println("SEIR: $(length(eqs)) ODEs")
for eq in eqs
    println("  ", eq)
end
```

```
SEIR: 7 ODEs
  Differential(t, 1)(pop_R(t)) ~ pop_I(t)*γ
  Differential(t, 1)(pop_I(t)) ~ -pop_I(t)*γ + pop_E(t)*σ
 Differential(t, 1)(pop_E(t)) ~ -pop_E(t)*σ + exp((-1 + θ(t))*κ)*phi_I(t)*β*κ*(1 -
ρ)
  Differential(t, 1)(phi_R(t)) ~ phi_I(t)*γ
  Differential(t, 1)(phi_I(t)) ~ phi_E(t)*σ - phi_I(t)*(β + γ)
 Differential(t, 1)(phi_E(t)) ~ (phi_E(t)*exp(0)*σ - exp((-1 + θ(t))*κ)*phi_I(t)*β*κ*(1 -
ρ)) / (-exp(0))
  Differential(t, 1)(θ(t)) ~ -phi_I(t)*β
```

The SEIR model has 5 ODEs: θ, φ_E, φ_I, φ_R, and R.

> [!NOTE]
>
> At the population level, the model tracks $S = \psi(\theta)$, R, and the combined "infected" fraction $E + I = 1 - S - R$. The edge-level variables φ_E and φ_I track the probability that a random neighbor is in each stage, providing more granular information than the population-level quantities.

```julia
σ_val = 0.2  # mean latent period = 5 days
ic_seir = default_initial_conditions(model_seir; seed_fraction = seed_fraction)
prob_seir = ODEProblem(
    model_seir.system,
    merge(ic_seir, Dict(β ⇒ β_val, γ ⇒ γ_val, σ ⇒ σ_val, κ ⇒ κ_val)),
    tspan,
)
sol_seir = solve(prob_seir, Tsit5(); saveat = 0.5)
```

```
retcode: Success
Interpolation: 1st order linear
t: 81-element Vector{Float64}:
  0.0
  0.5
  1.0
  1.5
  2.0
  2.5
  3.0
  3.5
  4.0
  4.5
```

```
 □
 36.0
 36.5
 37.0
 37.5
 38.0
 38.5
 39.0
 39.5
 40.0
u: 81-element Vector{Vector{Float64}}:
 [0.0, 0.0, 0.01, 0.0, 0.0, 0.010000000000000009, 1.0]
 [5.81984086357102e-5, 0.0008996512591088815, 0.009229032824458099, 5.6636401526874715e-
5, 0.0008634556651998017, 0.009229032824458108, 0.999962242398982]
 [0.0002182887755140383, 0.001639694725541273, 0.008824849244448799, 0.00020699040376162206
 [0.0004634856611399941, 0.0022680328648803983, 0.008682641980065214, 0.000428839849739497,
 [0.0007820756480752912, 0.0028187266411495377, 0.008729919172285125, 0.0007071119337892074
 [0.0011659556624731386, 0.00331618276894121, 0.008916641993600337, 0.0010317054622516737, 0
 [0.0016096410736669972, 0.0037779862034190053, 0.009208551328068974, 0.00139602452993727, 0
 [0.0021050734008383, 0.00421705813700124, 0.009582087781489196, 0.0017958654994891868, 0.0
 [0.0026633567036824 86, 0.004642879089875887, 0.010021526064601473, 0.002228792663107068, 0.
 [0.0032699774153242 07, 0.005062731636598094, 0.010516065815131308, 0.002693502420348 3984, 0
 □
 [0.27523121488789815, 0.07811065608392871, 0.10761823252342487, 0.18237507321205332, 0.0493
 [0.2850558930688857, 0.07906070067341141, 0.10787305414237072, 0.18857706467971846, 0.04982
 [0.2949924922331223, 0.07991769119478845, 0.10796811421498693, 0.19482813834577284, 0.05019
 [0.3050292778289908, 0.08067755145820128, 0.10790512291583035, 0.20112041564687114, 0.05050
 [0.3151543523836175, 0.08133599666203693, 0.10768784873031328, 0.20744629283859398, 0.05074
 [0.32535565550286843, 0.08188853339292773, 0.10732211845470385, 0.21379844099544523, 0.0509
 [0.3356204312699598, 0.08233195001272552, 0.10681325507124209, 0.22016882688250666, 0.05100
 [0.34593405589969756, 0.08266815809864742, 0.10615957788360722, 0.22654638008144304, 0.0510
 [0.3562829797693308, 0.0828966322435903, 0.10536537944971443, 0.23292215877052938, 0.050976
```

```julia
S_seir = compartment(sol_seir, model_seir, :S)
R_seir = compartment(sol_seir, model_seir, :R)
EI_seir = 1.0 .- S_seir .- R_seir  # E + I combined
```

```
81-element Vector{Float64}:
 0.010000000000000009
 0.010128684075262608
 0.010464543963436615
 0.010950674854890619
```

```
 0.011548645852730206
 0.01223282477930886
 0.012986537617558538
 0.013799145919390274
 0.014664405276611024
 0.015578797468860919
 ?
 0.18572972694161582
 0.1869343685025988
 0.1878860291031912
 0.18858252783326745
 0.18902349441443472
 0.18921027800192552
 0.1891448372446518
 0.1888273670875545
 0.18826164313980837
```

Comparing SIR and SEIR

```
# Build SIR for comparison
model_sir = build_sir(pgf, β, γ; form = :expanded)
ic_sir = default_initial_conditions(model_sir; seed_fraction = seed_fraction)
prob_sir = ODEProblem(
    model_sir.system,
    merge(ic_sir, Dict(β ⇒ β_val, γ ⇒ γ_val, κ ⇒ κ_val)),
    tspan,
)
sol_sir = solve(prob_sir, Tsit5(); saveat = 0.5)

S_sir = compartment(sol_sir, model_sir, :S)
I_sir = compartment(sol_sir, model_sir, :I)
R_sir = compartment(sol_sir, model_sir, :R)
```

```
81-element Vector{Float64}:
 0.0
 0.0014399021800027027
 0.003304790164502933 6
 0.005672864702126067
 0.00863740207248379
 0.012307952116939545
```

```
 0.016811057190712315
 0.022289842075696027
 0.02890284518272257
 0.0368194963915917
 □
 0.7960656312947881
 0.7964958825820977
 0.7968821762595986
 0.797228794589716
 0.797539615221106
 0.7978182242985102
 0.7980679164627564
 0.7982916948507572
 0.7984922710955112
```

```
plot(sol_sir.t, I_sir, label = "SIR (E+I)", linewidth = 2, color = :red)
plot!(sol_seir.t, EI_seir, label = "SEIR (E+I)", linewidth = 2, color = :orange,
      linestyle = :dash)
plot!(sol_sir.t, R_sir, label = "R (SIR)", linewidth = 1, color = :green)
plot!(sol_seir.t, R_seir, label = "R (SEIR)", linewidth = 1, color = :green,
      linestyle = :dash)
xlabel!("Time")
ylabel!("Fraction of population")
title!("SIR vs SEIR (σ=$σ_val, β=$β_val, γ=$γ_val)")
```

The exposed period delays the epidemic peak but does not change the final size (since R_0 depends only on the infectious period and transmission rate, not the latent period).

SIS model

In the SIS model, individuals recover to the susceptible state — there is no lasting immunity. This leads to an endemic equilibrium where $I > 0$ persists indefinitely.

```
model_sis = build_sis(pgf, β, γ)
eqs_sis = ModelingToolkit.equations(model_sis.system)
println("SIS: $(length(eqs_sis)) ODE")
for eq in eqs_sis
    println("  ", eq)
end
```

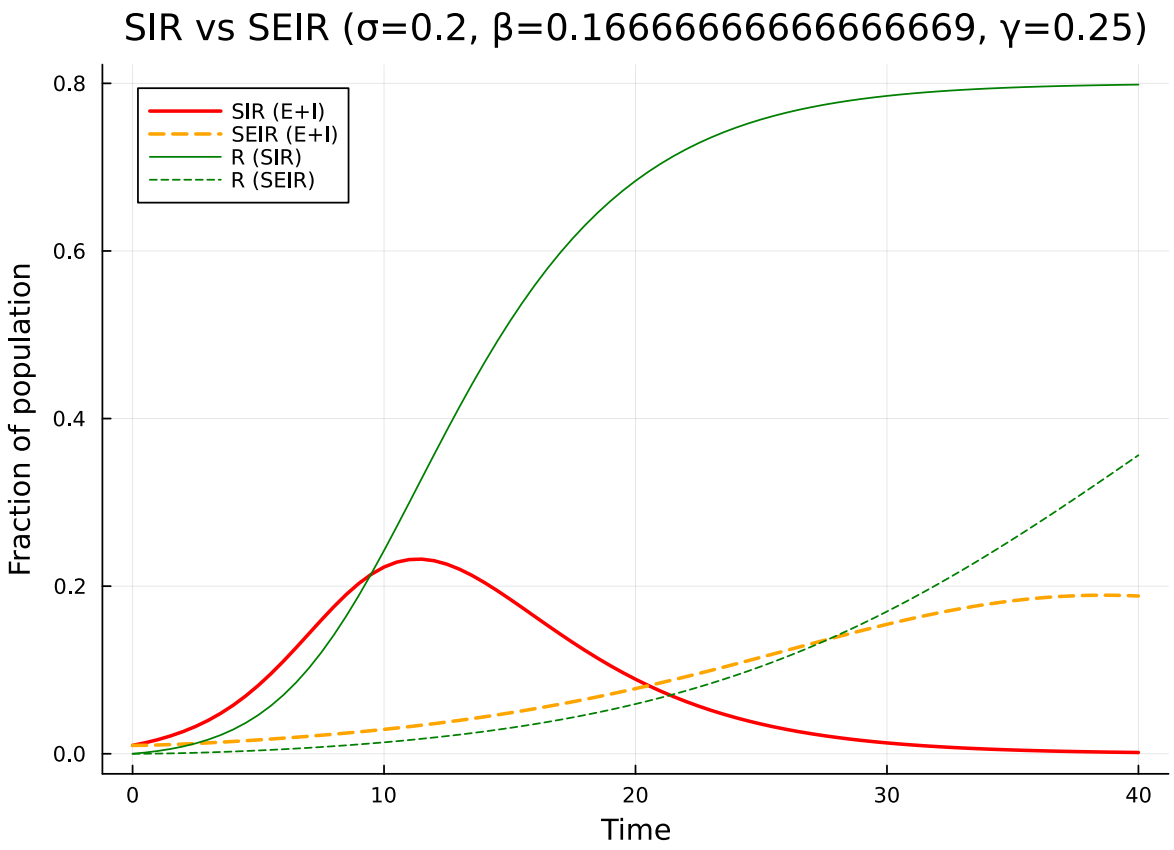

Figure 1: Figure 1: SIR vs SEIR: the exposed period delays and flattens the epidemic peak.

```
SIS: 1 ODE
  Differential(t, 1)(θ(t)) ~ ((θ(t)*exp(0) - exp((-1 + θ(t))*κ)*(1 - ρ))*β -
(1 - θ(t))*exp(0)*γ) / (-exp(0))
```

The SIS model has just 1 ODE for θ, with S and I as observables.

```
prob_sis = ODEProblem(
    model_sis.system,
    merge(
        default_initial_conditions(model_sis; seed_fraction = seed_fraction),
        Dict(β ⇒ β_val, γ ⇒ γ_val, κ ⇒ κ_val),
    ),
    (0.0, 80.0),
)
sol_sis = solve(prob_sis, Tsit5(); saveat = 0.5)

S_sis = compartment(sol_sis, model_sis, :S)
I_sis = compartment(sol_sis, model_sis, :I)
```

```
161-element Vector{Float64}:
 0.010000000000000009
 0.014564261464581452
 0.020122497266740424
 0.02686819215835068
 0.03501973204376374
 0.04481820577203155
 0.056532640439969084
 0.07043490689530896
 0.08678843645178125
 0.10582692954207096
 ?
 0.8002032808029537
 0.8002022275436793
 0.8002012121953894
 0.8002002831237899
 0.8001994935913623
 0.8001989017571696
 0.8001985706764562
 0.8001985682999876
 0.8001989674730821
```

```
plot(sol_sis.t, S_sis, label = "S", linewidth = 2, color = :blue)
plot!(sol_sis.t, I_sis, label = "I", linewidth = 2, color = :red)
xlabel!("Time")
ylabel!("Fraction of population")
title!("SIS on Poisson network (β=$β_val, γ=$γ_val, κ=$κ_val)")
```

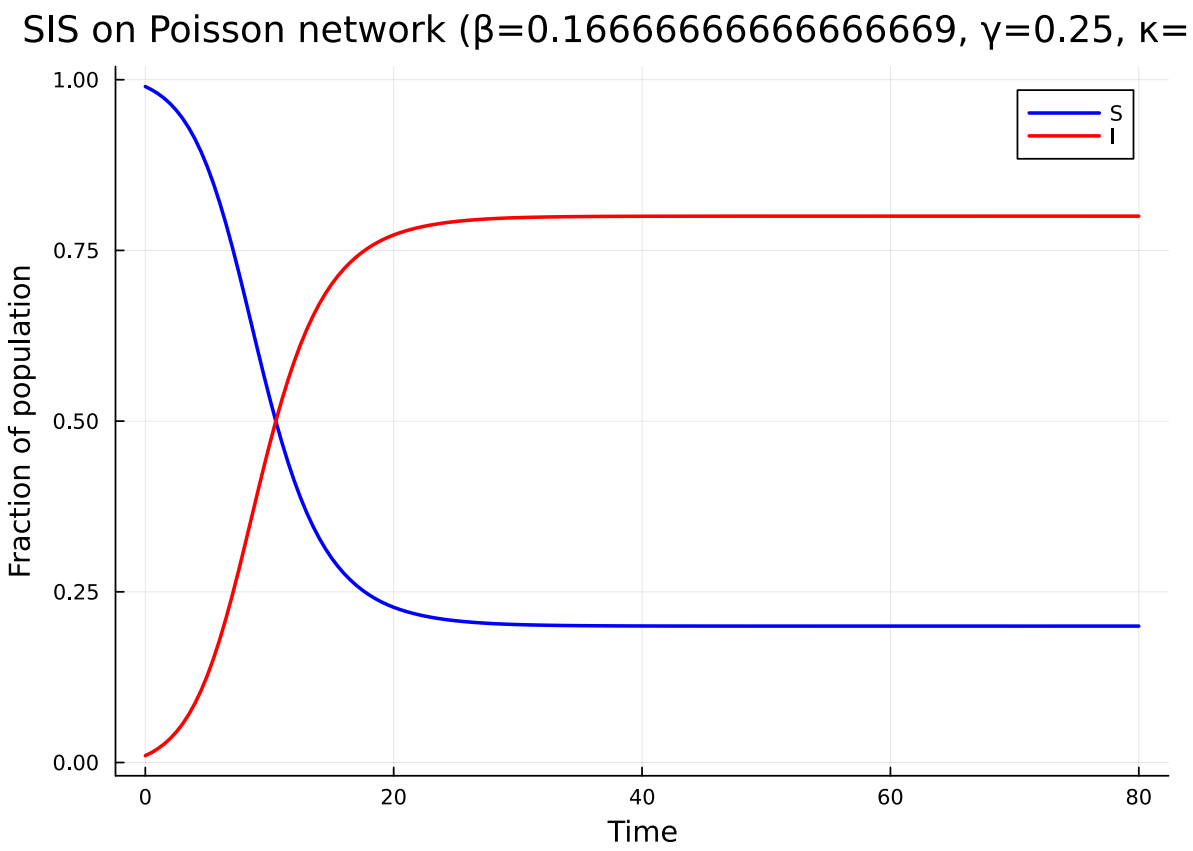

Figure 2: Figure 2: SIS model showing endemic equilibrium.

Unlike SIR, the infection reaches an endemic equilibrium where a constant fraction of the population remains infected.

Custom multi-stage infection

For diseases with multiple infectious stages (e.g., acute followed by chronic), we build a `DiseaseProgression` directly. Each stage can have a different transmission rate.

Consider a model where infection proceeds: $I_1 \xrightarrow{\gamma_1} I_2 \xrightarrow{\gamma_2} R$, with I_1 (acute) having transmission rate $\beta_1 = 0.8$ and I_2 (chronic) having $\beta_2 = 0.15$.

```
@parameters β₁ β₂ γ₁ γ₂

progression = DiseaseProgression(
    [
        DiseaseStage(:I1; transmission_rate = β₁),
        DiseaseStage(:I2; transmission_rate = β₂),
        DiseaseStage(:R; transmission_rate = 0),
    ],
    [
        DiseaseTransition(:I1, :I2, γ₁),
        DiseaseTransition(:I2, :R, γ₂),
    ];
    entry = :I1,
)

model_multi = build_edge_system(
    StaticConfigurationModel(pgf, progression);
    form = :expanded,
)
eqs_multi = ModelingToolkit.equations(model_multi.system)
println("Two-stage infection: $(length(eqs_multi)) ODEs")
for eq in eqs_multi
    println("  ", eq)
end
```

```
Two-stage infection: 7 ODEs
  Differential(t, 1)(pop_R(t)) ~ pop_I2(t)*γ₂
  Differential(t, 1)(pop_I2(t)) ~ pop_I1(t)*γ₁ - pop_I2(t)*γ₂
 Differential(t, 1)(pop_I1(t)) ~ -pop_I1(t)*γ₁ + (phi_I1(t)*β₁ + phi_I2(t)*β₂)*exp((-
1 + θ(t))*κ)*κ*(1 - ρ)
  Differential(t, 1)(phi_R(t)) ~ phi_I2(t)*γ₂
  Differential(t, 1)(phi_I2(t)) ~ phi_I1(t)*γ₁ - phi_I2(t)*(β₂ + γ₂)
 Differential(t, 1)(phi_I1(t)) ~ (phi_I1(t)*exp(0)*β₁ + phi_I1(t)*exp(0)*γ₁ -
(phi_I1(t)*β₁ + phi_I2(t)*β₂)*exp((-1 + θ(t))*κ)*κ*(1 - ρ)) / (-exp(0))
  Differential(t, 1)(θ(t)) ~ -phi_I1(t)*β₁ - phi_I2(t)*β₂
```

```
ic_multi = default_initial_conditions(model_multi)
prob_multi = ODEProblem(
    model_multi.system,
```

```
    merge(ic_multi, Dict(β₁ ⇒ 0.8, β₂ ⇒ 0.15, γ₁ ⇒ 0.5, γ₂ ⇒ 0.05, κ ⇒ κ_val)),
    (0.0, 150.0),
)
sol_multi = solve(prob_multi, Tsit5(); saveat = 0.5)
```

```
retcode: Success
Interpolation: 1st order linear
t: 301-element Vector{Float64}:
   0.0
   0.5
   1.0
   1.5
   2.0
   2.5
   3.0
   3.5
   4.0
   4.5
   ⋮
 146.0
 146.5
 147.0
 147.5
 148.0
 148.5
 149.0
 149.5
 150.0
u: 301-element Vector{Vector{Float64}}:
 [0.0, 0.0, 0.001, 0.0, 0.0, 0.0010000000000000009, 1.0]
 [5.738048418178316e-6, 0.0006074289067175093, 0.004733973840844085, 5.0558085954101205e-
6, 0.0005154203843888395, 0.003954467387176526, 0.999127802784181]
 [4.583505833618453e-5, 0.003185453329591618, 0.019448339327681252, 3.7474245116546296e-
5, 0.0025223625185648566, 0.015731753018128344, 0.9956119620662007]
 [0.00022441652068990175, 0.01325916696019162, 0.07333240253099463, 0.0001761319901947685
 [0.000903170729240297, 0.04711914583859462, 0.222428951980947, 0.0006899272561738214, 0.035
 [0.002978249120317911, 0.1274379239796133, 0.43151521543140076, 0.0021974886392826976, 0.09
 [0.007601579914419535, 0.24587525195305465, 0.5265440350163625, 0.005309001970863346, 0.157
 [0.015296837221577958, 0.36801177168754307, 0.4989659119019572, 0.009946689205931517, 0.208
 [0.02585967658645855, 0.4735555994579351, 0.4270722425810196, 0.015541075593219558, 0.23485
 [0.03879525727657989, 0.5577611702201124, 0.3508731402089691, 0.021528908396735817, 0.24160
 ⋮
```

```
 [0.9876738885352342, 0.0008468190786031996, 3.0861654490440233e-8, 0.09505005870732507, 2.6
7, -5.598019209810437e-7, 0.10652902493807835]
 [0.9876947964949997, 0.000825920504400283, 7.244536215166301e-9, 0.09505006675152178, 6.208
8, -1.3139193068841724e-7, 0.10652927320028574]
 [0.9877151880319962, 0.0008055430867335452, -2.8284168709941615e-8, 0.09505007885308271, -
2.424387725918524e-7, 5.131033097261013e-7, 0.10652964668305866]
 [0.9877350766291072, 0.0007856555527632234, -3.095757752557716e-8, 0.0950500797639488, -
2.6535873257492223e-7, 5.616119301540433e-7, 0.10652967479285336]
 [0.9877544746957445, 0.0007662460153893808, -2.091010479392662e-9, 0.09505006993190139, -
1.7943231613681883e-8, 3.7980751213520235e-8, 0.10652937134929744]
 [0.9877733936424823, 0.0007473201691063296, 1.5272067452544512e-8, 0.09505006401786578, 1.3
7, -2.7698809874620345e-7, 0.10652918882532265]
 [0.9877918450906275, 0.0007288712835294047, 8.824507847667446e-9, 0.09505006621383649, 7.56
8, -1.6003975474731168e-7, 0.10652925659638614]
 [0.9878098407147983, 0.0007108843556161479, -1.305778032012976e-8, 0.09505007366701898, -
1.1193010796222155e-7, 2.3689391747473266e-7, 0.1065294866181943]
 [0.9878273922429245, 0.0006933361096663089, -2.1315830680503827e-8, 0.09505007647982956, -
1.8271068798162344e-7, 3.8669432147142813e-7, 0.10652957342670286]
```

```
S_multi = compartment(sol_multi, model_multi, :S)
I_multi = compartment(sol_multi, model_multi, :I)
R_multi = compartment(sol_multi, model_multi, :R)
```

```
301-element Vector{Float64}:
 0.0
 5.738048418178316e-6
 4.583505833618453e-5
 0.00022441652068990175
 0.000903170729240297
 0.002978249120317911
 0.007601579914419535
 0.015296837221577958
 0.02585967658645855
 0.03879525727657989
 ?
 0.9876738885352342
 0.9876947964949997
 0.9877151880319962
 0.9877350766291072
 0.9877544746957445
 0.9877733936424823
 0.9877918450906275
```

```
0.9878098407147983
0.9878273922429245
```

```
plot(sol_multi.t, S_multi, label = "S", linewidth = 2, color = :blue)
plot!(sol_multi.t, I_multi, label = "I₁ + I₂", linewidth = 2, color = :red)
plot!(sol_multi.t, R_multi, label = "R", linewidth = 2, color = :green)
xlabel!("Time")
ylabel!("Fraction of population")
title!("Two-stage Infection (β₁=0.8, β₂=0.15)")
```

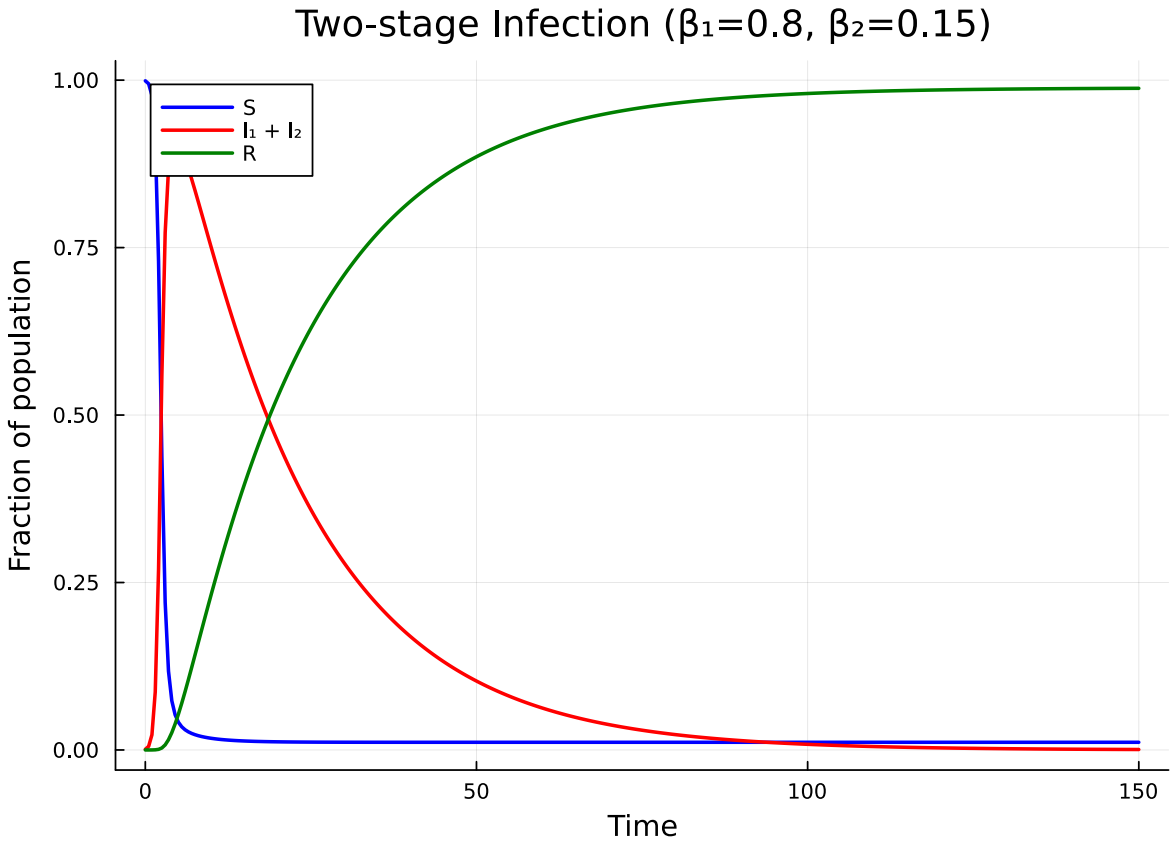

Figure 3: Figure 3: Two-stage infection with acute (high β) and chronic (low β) phases.

Edge-level dynamics

One of the unique features of edge-based models is access to the edge-level variables. These show the probability that a random neighbor is in each disease stage.

```
# Extract φ variables from the multi-stage solution
φ_vars = Dict{String,Any}()
for (name, var) in model_multi.variables
    if startswith(String(name), "φ")
        φ_vars[String(name)] = var
    end
end
println("Edge-level variables:")
for (name, _) in sort(collect(φ_vars); by = first)
    println("  $name")
end
```

```
Edge-level variables:
  φ_I1
  φ_I2
  φ_R
```

```
p = plot(xlabel = "Time", ylabel = "Edge probability",
         title = "Edge-level states (φ variables)")
for (name, var) in sort(collect(φ_vars); by = first)
    short = replace(name, r"\(t\)" ⇒ "")
    plot!(p, sol_multi.t, sol_multi[var], label = short, linewidth = 2)
end
p
```

Simulation validation

We compare each ODE prediction (SIR, SEIR, SIS) against Gillespie SSA on a Poisson network with the same mean degree $\kappa = 5$ via `NetworkOutbreaks.jl`.

```
include("../_validation.jl")

builder = poisson_graph_builder(1000, κ_val)

# SIR ribbon (epidemic, t ∈ [0,40])
t_sir, μ_sir, σ_sir = gillespie_ribbon(
    sir_model(), Dict(:β ⇒ β_val, :γ ⇒ γ_val), builder;
    N = 1000, n_graphs = 5, nsims_per_graph = 20,
    tspan = (0.0, 40.0), seed_fraction = seed_fraction)
```

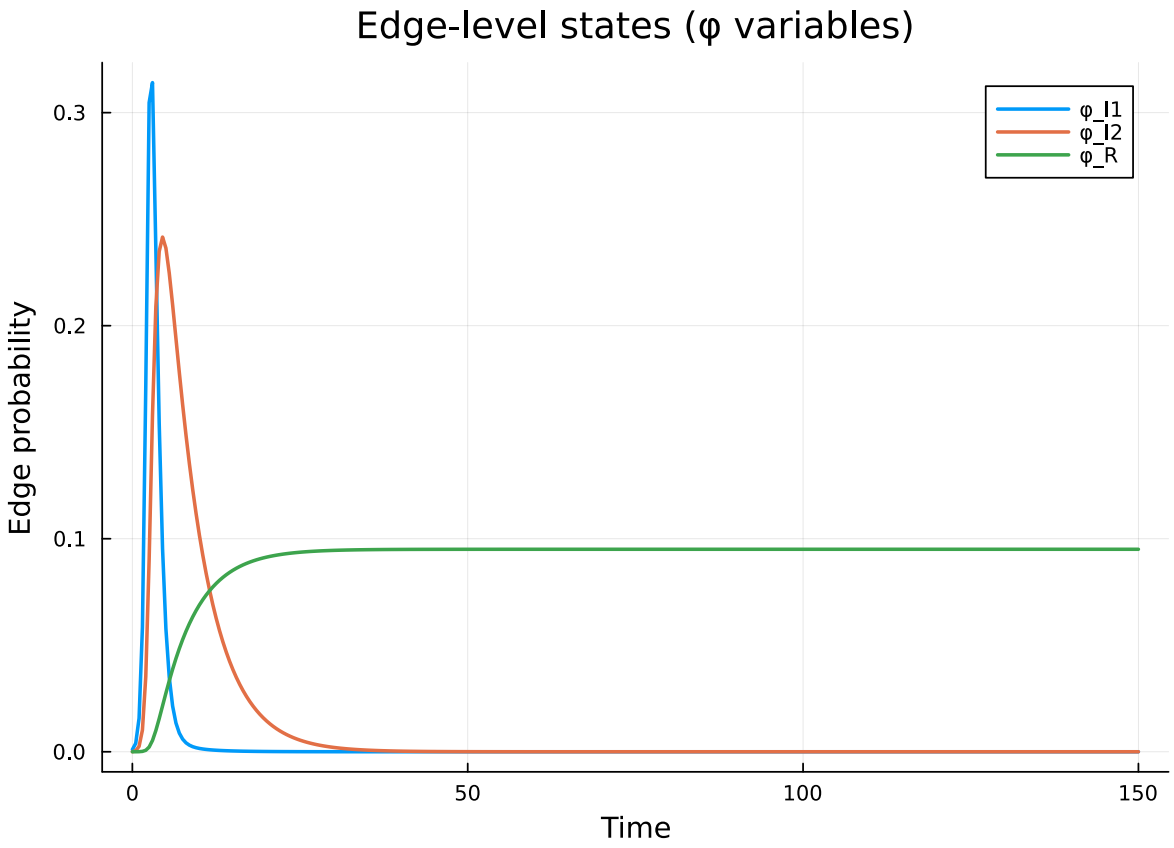

Figure 4: Figure 4: Edge-level probabilities for the two-stage infection model.

```
# SEIR ribbon
t_seir, μ_seir, σ_seir = gillespie_ribbon(
    seir_model(), Dict(:β ⇒ β_val, :γ ⇒ γ_val, :σ ⇒ σ_val), builder;
    N = 1000, n_graphs = 5, nsims_per_graph = 20,
    tspan = (0.0, 40.0), seed_fraction = seed_fraction, infected = :E)

# SIS ribbon (endemic, longer horizon)
t_sis_g, μ_sis_g, σ_sis_g = gillespie_ribbon(
    sis_model(), Dict(:β ⇒ β_val, :γ ⇒ γ_val), builder;
    N = 1000, n_graphs = 5, nsims_per_graph = 20,
    tspan = (0.0, 80.0), seed_fraction = seed_fraction)
```

```
([0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5 … 75.5, 76.0, 76.5, 77.0, 77.5, 78.0, 78.5,
```

```
EI_ssa = μ_seir[:E] .+ μ_seir[:I]
σEI_ssa = sqrt.(σ_seir[:E].^2 .+ σ_seir[:I].^2)
plot(t_sir, μ_sir[:I], ribbon = σ_sir[:I],
     label = "SSA SIR I (mean ± 1σ)", color = :red, fillalpha = 0.2, linealpha = 0.6)
plot!(sol_sir.t, I_sir, label = "EBCM SIR I", color = :red, linewidth = 2)
plot!(t_seir, EI_ssa, ribbon = σEI_ssa,
     label = "SSA SEIR E+I (mean ± 1σ)", color = :orange, fillalpha = 0.2, linealpha = 0.6
plot!(sol_seir.t, EI_seir, label = "EBCM SEIR E+I", color = :orange, linewidth = 2)
xlabel!("Time"); ylabel!("Fraction infected")
title!("EBCM vs SSA on Poisson κ=5")
```

```
plot(t_sis_g, μ_sis_g[:I], ribbon = σ_sis_g[:I],
     label = "SSA SIS (mean ± 1σ)", color = :red, fillalpha = 0.2, linealpha = 0.6)
plot!(sol_sis.t, I_sis, label = "EBCM SIS", color = :red, linewidth = 2)
xlabel!("Time"); ylabel!("Fraction infected")
title!("SIS endemic equilibrium")
```

The deterministic EBCM curves track the SSA ensemble means well across SIR, SEIR, and SIS, confirming that the configuration-model closure recovers the correct large-network limit.

Summary

- SIS: 1 ODE — produces endemic equilibria.
- Multi-stage: arbitrary disease graphs via `DiseaseProgression` with per-stage transmission rates.

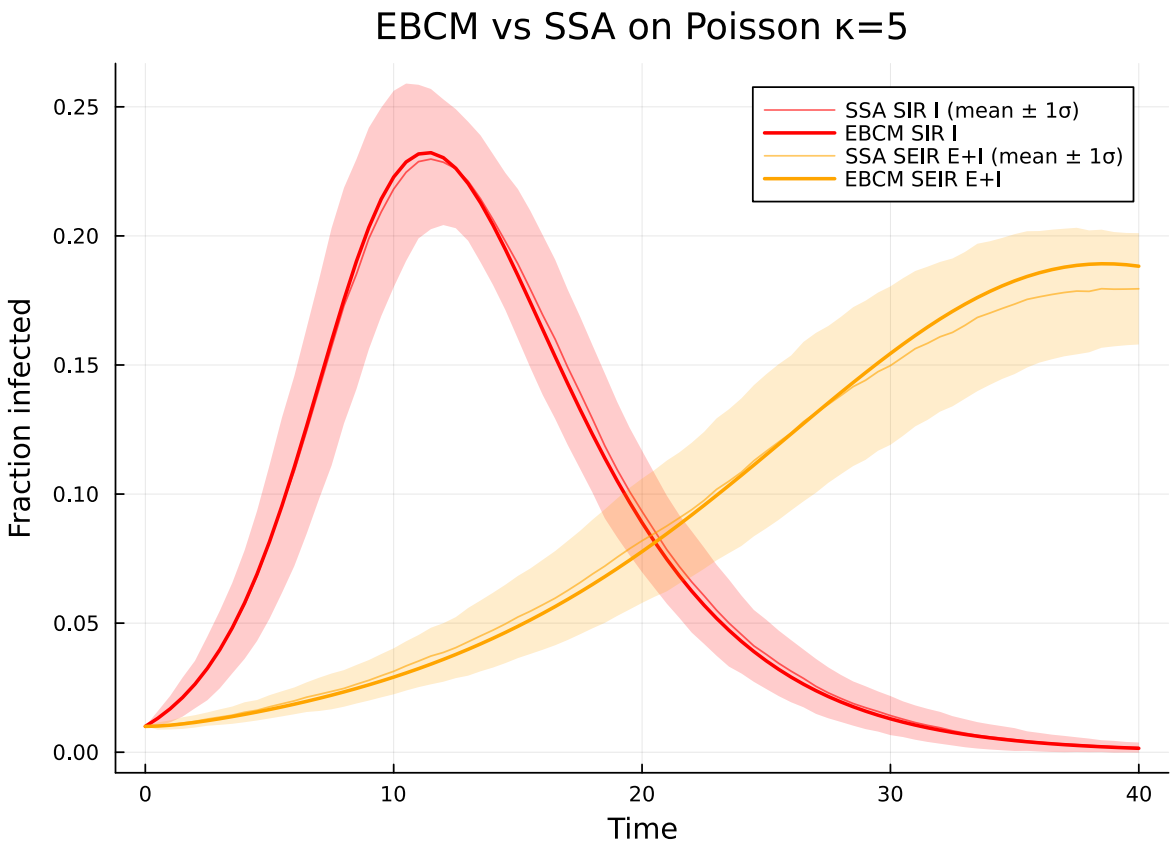

Figure 5: Figure 5: Gillespie SSA mean ± 1σ ribbons (matched line colors) versus EBCM I/(E+I) curve for SIR and SEIR.

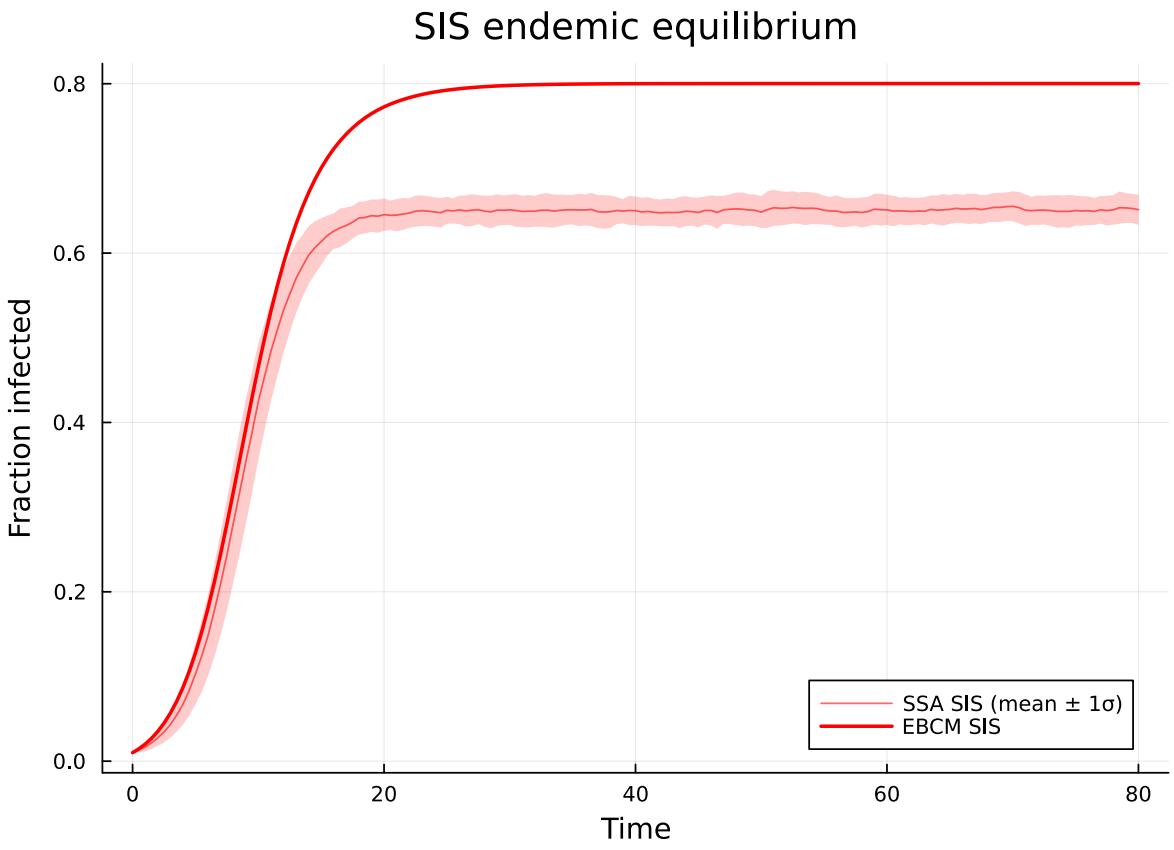

Figure 6: Figure 6: SIS on Poisson κ=5: EBCM endemic equilibrium versus SSA mean $\pm$ 1σ (red ribbon).

- The package automates all the bookkeeping for edge-level variables, regardless of disease complexity.